

# 2D Systolic Array

## Complete Code Explanation

---

Line-by-Line Verilog Code Walkthrough  
For EDGE-AI 2026 Hackathon Preparation

Files covered: 8 Verilog modules (from PE to full system)

Tools: Icarus Verilog (iverilog) + GTKWave

## Table of Contents

| # | File                | Module                  | Description                 |
|---|---------------------|-------------------------|-----------------------------|
| 1 | 1_pe.v              | Processing Element (PE) | The basic building block    |
| 2 | 2_pe_tb.v           | PE Testbench            | Tests a single PE           |
| 3 | 3_systolic_array.v  | N x N Systolic Array    | Wires $N^2$ PEs into a grid |
| 4 | 4_systolic_2x2_tb.v | 2x2 Array Testbench     | Manual skewing test         |
| 5 | 5_skew_buffer.v     | Skew Buffer             | Automatic input staggering  |
| 6 | 6_controller.v      | Controller FSM          | State machine for control   |
| 7 | 7_systolic_top.v    | Top-Level Wrapper       | Connects all pieces         |
| 8 | 8_systolic_4x4_tb.v | 4x4 System Testbench    | Full system verification    |

## How to Read This Document

Start from File 1 and work your way down. Each file builds on the previous ones. Don't skip ahead - the concepts stack on top of each other.

For each file, you'll see:

- What the module does (plain English)
- The actual Verilog source code
- Line-by-line explanation of every important construct

## File 1: 1\_pe.v - Processing Element

This is the MOST IMPORTANT file. Everything else is built from this.

### What does a PE do?

Think of a PE as a tiny calculator that does exactly 3 things every clock cycle:

1. Multiply two numbers:  $\text{product} = a_{\text{in}} \times b_{\text{in}}$
2. Add to running total:  $\text{acc} = \text{acc} + \text{product}$
3. Pass the numbers along:  $a_{\text{out}} = a_{\text{in}}, b_{\text{out}} = b_{\text{in}}$

That's it. That's the entire PE.

### Source Code

```
// =====
// PE.v -- Processing Element for a Systolic Array
// =====
// This is the basic building block. Each PE does ONE thing:
//   acc = acc + (a_in * b_in)
// and passes a_in to the right, b_in downward.
// =====

module PE #(
    parameter DATA_WIDTH = 8          // Width of each input (a and b)
)(
    input  wire clk,                   // Clock
    input  wire rst,                   // Synchronous reset (active high)
    input  wire en,                    // Enable signal

    // --- Data flowing LEFT -> RIGHT (matrix A values) ---
    input  wire signed [DATA_WIDTH-1:0] a_in,    // From left neighbor
    output reg signed [DATA_WIDTH-1:0] a_out,    // To right neighbor

    // --- Data flowing TOP -> BOTTOM (matrix B values) ---
    input  wire signed [DATA_WIDTH-1:0] b_in,    // From top neighbor
    output reg signed [DATA_WIDTH-1:0] b_out,    // To bottom neighbor

    // --- Accumulated result (stays in this PE) ---
    // 2*DATA_WIDTH bits to prevent overflow
    // e.g., 8-bit x 8-bit = 16-bit product, accumulated over N cycles
    output reg signed [2*DATA_WIDTH-1:0] acc
);

always @(posedge clk) begin
    if (rst) begin
        // Reset: clear everything
        a_out <= 0;
        b_out <= 0;
        acc  <= 0;
    end
    else if (en) begin
        // ---- THE CORE OPERATION ----
```

```

        // Multiply a_in and b_in, add to running total
        acc <= acc + (a_in * b_in);

        // ---- PASS-THROUGH ----
        // Forward a_in to the right neighbor (delayed by 1 clock)
        a_out <= a_in;
        // Forward b_in to the bottom neighbor (delayed by 1 clock)
        b_out <= b_in;
    end
    // If en=0: all registers hold their current value
end
endmodule

```

## Line-by-Line Explanation

### Module Declaration (Lines 9-28)

module PE #(parameter DATA\_WIDTH = 8)

- 'module PE' - We're defining a hardware block called PE
- 'parameter DATA\_WIDTH = 8' - Each number is 8 bits wide (holds -128 to +127)
- The #(...) syntax means 'this module has configurable settings'

### Ports Explained

- clk - The heartbeat. Everything happens on the rising edge.
- rst - When HIGH, clears everything to zero. Like a 'clear' button.
- en - When HIGH, the PE computes. When LOW, it holds values.

- a\_in - A number coming from the LEFT neighbor
- a\_out - That same number, passed to the RIGHT neighbor (1 clock later)
- b\_in - A number coming from the TOP neighbor
- b\_out - That same number, passed to the BOTTOM neighbor (1 clock later)

- acc - The accumulator (running total). 16 bits wide because:  
8-bit x 8-bit = 16-bit product. Using only 8 bits would overflow!

### wire vs reg

- 'wire' = input (someone else drives this signal)
- 'reg' = output (we drive it from an always block)
- 'signed' = the number can be negative

### The always Block (Lines 30-49)

always @(posedge clk)

Means: 'Do this on every RISING EDGE of the clock'

if (rst) begin ...clear everything... end

If reset is HIGH: set a\_out, b\_out, acc all to zero.

else if (en) begin

```
acc <= acc + (a_in * b_in); -- THE CORE OPERATION (MAC)
a_out <= a_in;              -- Pass a_in to right neighbor
b_out <= b_in;              -- Pass b_in to bottom neighbor
end
```

If en=0: nothing happens. All registers keep their current values.

The '<=' is NON-BLOCKING assignment - all three assignments happen 'at the same time' (standard for sequential logic in Verilog).

**KEY: A PE = 1 multiplier + 1 adder + 3 registers. That's all!**

## File 2: 2\_pe\_tb.v - PE Testbench

A testbench is NOT hardware. It's a simulation script.

### What is a Testbench?

A testbench does 4 things:

1. Creates a fake clock signal
2. Creates an instance of the module we want to test
3. Feeds it input values
4. Checks if the outputs are correct

Testbench modules have NO PORTS (no inputs/outputs). They're self-contained.

### Source Code

```
// =====
// pe_tb.v -- Testbench for a SINGLE Processing Element
// =====
// This testbench feeds a_in and b_in values into one PE
// and shows you how it accumulates the result cycle by cycle.
//
// We simulate what PE[0][0] does during a 3x3 matrix multiply:
//   Cycle 0: a_in=1, b_in=7   -> acc = 0 + 1*7 = 7
//   Cycle 1: a_in=2, b_in=10  -> acc = 7 + 2*10 = 27
//   Cycle 2: a_in=3, b_in=13  -> acc = 27 + 3*13 = 66
//
// Expected final acc = 66 = C[0][0] [ok]
// =====

`timescale 1ns / 1ps

module pe_tb;

    // ---- Parameters ----
    parameter DATA_WIDTH = 8;
    parameter CLK_PERIOD = 10;      // 10ns clock -> 100MHz

    // ---- Signals ----
    reg clk, rst, en;
    reg signed [DATA_WIDTH-1:0] a_in, b_in;
    wire signed [DATA_WIDTH-1:0] a_out, b_out;
    wire signed [2*DATA_WIDTH-1:0] acc;

    // ---- Instantiate the PE (Unit Under Test) ----
    PE #(.DATA_WIDTH(DATA_WIDTH)) uut (
        .clk(clk),
        .rst(rst),
        .en(en),
        .a_in(a_in),
        .b_in(b_in),
        .a_out(a_out),
        .b_out(b_out),
        .acc(acc)
    );
endmodule
```

```

);

// ---- Clock Generator ----
// Toggles every 5ns -> 10ns period -> 100MHz
initial clk = 0;
always #(CLK_PERIOD/2) clk = ~clk;

// ---- Dump waveforms for GTKWave ----
initial begin
    $dumpfile("pe_tb.vcd");    // Output file for GTKWave
    $dumpvars(0, pe_tb);      // Dump ALL signals in this module
end

// ---- Test Stimulus ----
initial begin
    // =====
    // PHASE 1: Reset
    // =====
    $display("=====");
    $display(" Single PE Testbench");
    $display(" Simulating PE[0][0] of 3x3 matmul");
    $display("=====");

    rst = 1;    // Assert reset
    en = 0;     // Disable computation
    a_in = 0;
    b_in = 0;

    // Hold reset for 2 clock cycles
    @(posedge clk);
    @(posedge clk);
    rst = 0;    // Release reset
    en = 1;     // Enable computation

    $display("\nTime | a_in | b_in | a_in*b_in | acc (expected)");
    $display("-----|-----|-----|-----|-----");

    // =====
    // PHASE 2: Feed data (3 cycles)
    // =====

    // Cycle 0: A[0][0]=1, B[0][0]=7
    a_in = 8'sd1;    // sd = signed decimal
    b_in = 8'sd7;
    @(posedge clk);
    #1; // Small delay so $display reads the updated values
    $display("%4t | %2d | %2d | %4d | %4d (exp: 7)",
        $time, a_in, b_in, a_in*b_in, acc);

    // Cycle 1: A[0][1]=2, B[1][0]=10
    a_in = 8'sd2;
    b_in = 8'sd10;
    @(posedge clk);
    #1;
    $display("%4t | %2d | %2d | %4d | %4d (exp: 27)",
        $time, a_in, b_in, a_in*b_in, acc);

    // Cycle 2: A[0][2]=3, B[2][0]=13
    a_in = 8'sd3;
    b_in = 8'sd13;
    @(posedge clk);

```

```
#1;
$display("%4t | %2d | %2d | %4d | %4d (exp: 66)",
        $time, a_in, b_in, a_in*b_in, acc);

// =====
// PHASE 3: Disable and check
// =====
en = 0;          // Stop computing
a_in = 0;
b_in = 0;

// Wait a couple more cycles to see that acc holds its value
@(posedge clk);
@(posedge clk);

$display("\n=====");
if (acc == 66) begin
    $display(" [ok] PASS! Final acc = %0d (expected 66)", acc);
    $display(" C[0][0] = 1*7 + 2*10 + 3*13 = 66");
end else begin
    $display(" [X] FAIL! Final acc = %0d (expected 66)", acc);
end
$display("=====");

// Also verify pass-through worked
$display("\nPass-through check:");
$display(" Last a_out = %0d (should be 3 = last a_in)", a_out);
$display(" Last b_out = %0d (should be 13 = last b_in)", b_out);

$finish;
end

endmodule
```

## Key Constructs Explained

### `timescale 1ns / 1ps

Tells the simulator: '1 time unit = 1 nanosecond, precision = 1 picosecond'  
So when we write #10, it means 'wait 10 nanoseconds'.

### Clock Generator

```
initial clk = 0;
always #(CLK_PERIOD/2) clk = ~clk;
```

- Clock starts LOW
- Every 5ns, flip it. Creates a 10ns period (100MHz) square wave:

```
clk: __/--\__/--\__/--\__
      0 5 10 15 20 25
```

### \$dumpfile and \$dumpvars

\$dumpfile('pe\_tb.vcd') - Create a waveform file for GTKWave  
\$dumpvars(0, pe\_tb) - Record ALL signals (the 0 means all levels deep)

### @(posedge clk)

Means: 'Wait for the next rising edge of the clock'



This is how we synchronize our test stimulus with the clock.

### 8'sd1

An 8-bit Signed Decimal value of 1.

8 = 8 bits wide

's = signed

d = decimal (base 10)

1 = the value

### What This Test Does

Feeds 3 pairs of values into the PE and checks accumulation:

Cycle 0:  $a_{in}=1, b_{in}=7 \rightarrow acc = 0 + 1 \times 7 = 7$

Cycle 1:  $a_{in}=2, b_{in}=10 \rightarrow acc = 7 + 2 \times 10 = 27$

Cycle 2:  $a_{in}=3, b_{in}=13 \rightarrow acc = 27 + 3 \times 13 = 66$

This simulates PE[0][0] computing  $C[0][0] = 66$  in a 3x3 matrix multiply.

**RUN IT: iverilog -o pe\_test.vvp 1\_pe.v 2\_pe\_tb.v && vvp pe\_test.vvp**

## File 3: 3\_systolic\_array.v - N x N Array

Takes  $N^2$  copies of PE and wires them into a grid.

### What This Does

Creates an  $N \times N$  grid of PEs where:

- A values flow LEFT to RIGHT (each PE passes to its right neighbor)
- B values flow TOP to BOTTOM (each PE passes to its bottom neighbor)
- Each PE accumulates:  $acc = acc + a_{in} * b_{in}$

After computation,  $PE[i][j]$  holds the result  $C[i][j]$ .

### Source Code

```
// =====
// systolic_array.v -- NxN 2D Systolic Array
// =====
// This module creates an NxN grid of Processing Elements (PEs)
// and wires them together:
//   - a_in flows LEFT -> RIGHT (each PE passes to its right neighbor)
//   - b_in flows TOP -> BOTTOM (each PE passes to its bottom neighbor)
//   - Each PE accumulates: acc = acc + a_in * b_in
//
// After 2N-1 clock cycles, each PE[i][j] holds C[i][j].
//
// IMPORTANT: This module expects ALREADY SKEWED inputs!
// (The skew buffers handle that -- we'll add them later)
// =====

module systolic_array #(
    parameter N = 4,                // Array dimension (NxN PEs)
    parameter DATA_WIDTH = 8       // Bit width of each input value
)(
    input  wire clk,
    input  wire rst,
    input  wire en,

    // --- Inputs from the LEFT edge (one per row) ---
    // a_in[0] feeds row 0, a_in[1] feeds row 1, etc.
    input  wire signed [DATA_WIDTH-1:0] a_in [0:N-1],

    // --- Inputs from the TOP edge (one per column) ---
    // b_in[0] feeds column 0, b_in[1] feeds column 1, etc.
    input  wire signed [DATA_WIDTH-1:0] b_in [0:N-1],

    // --- Results: one per PE ---
    // result[i][j] = C[i][j] after computation completes
    output wire signed [2*DATA_WIDTH-1:0] result [0:N-1][0:N-1]
);

// =====
// INTERNAL WIRES
// =====
// These carry data BETWEEN PEs.
```

```

//
// a_wire[i][j] = the 'a' value entering PE[i][j] from the left
//   - a_wire[i][0] = a_in[i] (external input, left edge)
//   - a_wire[i][j] = a_out of PE[i][j-1] (for j > 0)
//   - a_wire[i][N] = unused (falls off the right edge)
//
// b_wire[i][j] = the 'b' value entering PE[i][j] from the top
//   - b_wire[0][j] = b_in[j] (external input, top edge)
//   - b_wire[i][j] = b_out of PE[i-1][j] (for i > 0)
//   - b_wire[N][j] = unused (falls off the bottom edge)

wire signed [DATA_WIDTH-1:0] a_wire [0:N-1][0:N]; // N rows, N+1 columns
wire signed [DATA_WIDTH-1:0] b_wire [0:N][0:N-1]; // N+1 rows, N columns

// =====
// CONNECT EXTERNAL INPUTS TO THE EDGES
// =====
genvar idx;
generate
    for (idx = 0; idx < N; idx = idx + 1) begin : connect_inputs
        // A values enter from the LEFT edge (column 0 of a_wire)
        assign a_wire[idx][0] = a_in[idx];

        // B values enter from the TOP edge (row 0 of b_wire)
        assign b_wire[0][idx] = b_in[idx];
    end
endgenerate

// =====
// GENERATE THE NxN GRID OF PEs
// =====
// This is where the magic happens!
// We use nested generate loops to create N^2 PEs and wire them.
//
// Each PE[i][j]:
//   - Reads a_in from a_wire[i][j]      (from left neighbor or edge)
//   - Reads b_in from b_wire[i][j]      (from top neighbor or edge)
//   - Writes a_out to a_wire[i][j+1]    (to right neighbor)
//   - Writes b_out to b_wire[i+1][j]    (to bottom neighbor)
//   - Stores result in result[i][j]

genvar i, j;
generate
    for (i = 0; i < N; i = i + 1) begin : row
        for (j = 0; j < N; j = j + 1) begin : col
            PE #(
                .DATA_WIDTH(DATA_WIDTH)
            ) pe_inst (
                .clk    (clk),
                .rst    (rst),
                .en      (en),

                // A flows left -> right
                .a_in   (a_wire[i][j]),      // from left
                .a_out   (a_wire[i][j+1]),    // to right

                // B flows top -> bottom
                .b_in   (b_wire[i][j]),      // from top
                .b_out   (b_wire[i+1][j]),    // to bottom

                // Accumulated result

```

```

        .acc    (result[i][j])
    );
    end
end
endgenerate

// =====
// That's it! The wiring is automatic thanks to generate.
//
// Visually for N=3:
//
//    a_in[0]-->[PE00]-->[PE01]-->[PE02]-->(unused)
//           |           |           |
//    a_in[1]-->[PE10]-->[PE11]-->[PE12]-->(unused)
//           |           |           |
//    a_in[2]-->[PE20]-->[PE21]-->[PE22]-->(unused)
//           |           |           |
//           (unused) (unused) (unused)
//           ^           ^           ^
//        b_in[0] b_in[1] b_in[2]
//        (fed from top)
// =====

endmodule

```

## Key Concept: Internal Wires

wire a\_wire [0:N-1][0:N] -- N rows, N+1 columns

wire b\_wire [0:N][0:N-1] -- N+1 rows, N columns

Think of these as PIPES between PEs:

a\_wire[i][j] = the 'a' value entering PE[i][j] from the left

b\_wire[i][j] = the 'b' value entering PE[i][j] from the top

Why N+1? Because PE[i][N-1]'s output goes to a\_wire[i][N], which is the right edge - it goes nowhere (unused).

## Key Concept: generate Block

generate + for is NOT a runtime loop!

It tells the COMPILER: 'Create N^2 PE instances at compile time.'

For N=4, this creates 16 PEs and wires them automatically.

Each PE[i][j] is wired:

.a\_in (a\_wire[i][j]) -- receives from left

.a\_out (a\_wire[i][j+1]) -- sends to right

.b\_in (b\_wire[i][j]) -- receives from top

.b\_out (b\_wire[i+1][j]) -- sends to bottom

Visual for N=3:

```

a_in[0] --> [PE00] --> [PE01] --> [PE02] --> (unused)
           |           |           |

```

---

## 2D Systolic Array - Complete Code Explanation

```
a_in[1] --> [PE10] --> [PE11] --> [PE12] --> (unused)
      |           |           |
a_in[2] --> [PE20] --> [PE21] --> [PE22] --> (unused)
      |           |           |
      (unused)    (unused)    (unused)
      ^           ^           ^
      b_in[0]     b_in[1]     b_in[2]
```

## File 4: 4\_systolic\_2x2\_tb.v - 2x2 Test

Tests a 2x2 array with MANUALLY SKEWED inputs.

### The Test

Multiplies:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

### Why Skewing is Needed

We can't feed all data at once. We STAGGER (skew) the inputs so the right values meet at the right PE at the right time.

CYCLE 0: a\_in = [1, 0] b\_in = [5, 0] -- Row/Col 0 starts

CYCLE 1: a\_in = [2, 3] b\_in = [7, 6] -- Both active

CYCLE 2: a\_in = [0, 4] b\_in = [0, 8] -- Row/Col 0 done

Why this works:

PE[0][0] sees pairs (1,5), (2,7) -> acc = 5+14 = 19

PE[0][1] sees pairs (1,6), (2,8) -> acc = 6+16 = 22

(The '1' arrives 1 cycle late because it passed through PE[0][0]!)

(The '6' is also fed 1 cycle late from the skewed input!)

So they meet at the right time!

### Source Code

```
// =====
// systolic_2x2_tb.v -- Testbench for 2x2 Systolic Array
// =====
// We multiply:
//   A = |1  2|   B = |5  6|
//       |3  4|       |7  8|
//
//   C = A x B = |1*5+2*7  1*6+2*8| = |19  22|
//               |3*5+4*7  3*6+4*8|   |43  50|
//
// This testbench MANUALLY feeds skewed inputs so you can
// see exactly how data flows through the array.
//
// Skewed A (fed from left):      Skewed B (fed from top):
//   Cycle:  0  1  2              Cycle:  0  1  2
//   Row 0:  1  2  0              Col 0:  5  7  0
//   Row 1:  0  3  4              Col 1:  0  6  8
//
// Timeline of what each PE computes:
//   Cycle 0: PE00: acc=0+1*5=5    PE01: idle    PE10: idle    PE11: idle
//   Cycle 1: PE00: acc=5+2*7=19[ok] PE01: 1*6=6    PE10: 3*5=15    PE11: idle
//   Cycle 2: PE00: done           PE01: 6+2*8=22[ok] PE10: 15+4*7=43[ok] PE11: 3*6=18
//   Cycle 3:                      PE11: 18+4*8=50[ok]
```

```
// =====

`timescale 1ns / 1ps

module systolic_2x2_tb;

    // ---- Parameters ----
    parameter N = 2;
    parameter DATA_WIDTH = 8;
    parameter CLK_PERIOD = 10;

    // Total compute cycles needed = 2*N - 1 = 3 for 2x2
    parameter COMPUTE_CYCLES = 2*N - 1;

    // ---- Signals ----
    reg clk, rst, en;
    reg signed [DATA_WIDTH-1:0] a_in [0:N-1];
    reg signed [DATA_WIDTH-1:0] b_in [0:N-1];
    wire signed [2*DATA_WIDTH-1:0] result [0:N-1][0:N-1];

    // ---- Instantiate the 2x2 Systolic Array ----
    systolic_array #(
        .N(N),
        .DATA_WIDTH(DATA_WIDTH)
    ) uut (
        .clk(clk),
        .rst(rst),
        .en(en),
        .a_in(a_in),
        .b_in(b_in),
        .result(result)
    );

    // ---- Clock Generator ----
    initial clk = 0;
    always #(CLK_PERIOD/2) clk = ~clk;

    // ---- Dump waveforms for GTKWave ----
    initial begin
        $dumpfile("systolic_2x2_tb.vcd");
        $dumpvars(0, systolic_2x2_tb);
    end

    // ---- Expected Results ----
    // C = A x B = |19 22|
    //               |43 50|

    // ---- Test Stimulus ----
    integer cycle;
    initial begin
        $display("=====");
        $display(" 2x2 Systolic Array Testbench");
        $display(" A = |1 2|  B = |5 6|  C = |19 22|");
        $display("      |3 4|      |7 8|      |43 50|");
        $display("=====\\n");

        // ---- RESET ----
        rst = 1;
        en = 0;
        a_in[0] = 0; a_in[1] = 0;
        b_in[0] = 0; b_in[1] = 0;
    end
endmodule
```

```

@(posedge clk);
@(posedge clk);
rst = 0;
en = 1;

$display("Feeding SKEWED inputs into the array:\n");
$display("Cycle | a_in[0] a_in[1] | b_in[0] b_in[1] | PE00.acc PE01.acc PE10.acc PE11.acc");
$display("-----|-----|-----|-----");

// =====
// CYCLE 0: Feed first values
// =====
// A row 0 starts: a[0][0]=1, A row 1 delayed: 0
// B col 0 starts: b[0][0]=5, B col 1 delayed: 0
a_in[0] = 8'sd1; a_in[1] = 8'sd0;
b_in[0] = 8'sd5; b_in[1] = 8'sd0;
@(posedge clk);
#1;
$display(" 0 | %2d %2d | %2d %2d | %4d %4d %4d %4d",
        a_in[0], a_in[1], b_in[0], b_in[1],
        result[0][0], result[0][1], result[1][0], result[1][1]);

// =====
// CYCLE 1: Second values
// =====
// A row 0: a[0][1]=2, A row 1 starts: a[1][0]=3
// B col 0: b[1][0]=7, B col 1 starts: b[0][1]=6
a_in[0] = 8'sd2; a_in[1] = 8'sd3;
b_in[0] = 8'sd7; b_in[1] = 8'sd6;
@(posedge clk);
#1;
$display(" 1 | %2d %2d | %2d %2d | %4d %4d %4d %4d",
        a_in[0], a_in[1], b_in[0], b_in[1],
        result[0][0], result[0][1], result[1][0], result[1][1]);

// =====
// CYCLE 2: Last values
// =====
// A row 0: done (0), A row 1: a[1][1]=4
// B col 0: done (0), B col 1: b[1][1]=8
a_in[0] = 8'sd0; a_in[1] = 8'sd4;
b_in[0] = 8'sd0; b_in[1] = 8'sd8;
@(posedge clk);
#1;
$display(" 2 | %2d %2d | %2d %2d | %4d %4d %4d %4d",
        a_in[0], a_in[1], b_in[0], b_in[1],
        result[0][0], result[0][1], result[1][0], result[1][1]);

// =====
// Wait 1 more cycle for last PE to finish
// =====
a_in[0] = 0; a_in[1] = 0;
b_in[0] = 0; b_in[1] = 0;
@(posedge clk);
#1;

// ---- Disable and check results ----
en = 0;

$display("\n=====");

```



```
$display(" RESULTS:");
$display(" C[0][0] = %0d (expected 19)", result[0][0]);
$display(" C[0][1] = %0d (expected 22)", result[0][1]);
$display(" C[1][0] = %0d (expected 43)", result[1][0]);
$display(" C[1][1] = %0d (expected 50)", result[1][1]);
$display("=====");

// ---- Verify ----
if (result[0][0] == 19 && result[0][1] == 22 &&
    result[1][0] == 43 && result[1][1] == 50) begin
    $display(" >>> ALL PASS! Matrix multiplication correct! <<<");
end else begin
    $display(" >>> FAIL! Check the waveform in GTKWave <<<");
end
$display("=====");

// Wait a bit then finish
@(posedge clk);
@(posedge clk);
$finish;
end

endmodule
```

**RUN: iverilog -g2012 -o test.vvp 1\_pe.v 3\_systolic\_array.v 4\_systolic\_2x2\_tb.v**

## File 5: 5\_skew\_buffer.v - Skew Buffer

Automates the input staggering using shift registers.

### The Problem

In File 4, we manually typed the skewed inputs. That's fine for testing but impractical for a real system. The skew buffer does this automatically.

### How It Works: Shift Registers

Think of each row as a QUEUE (line of people waiting).

For N=3, Matrix A =  $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

After loading, the shift registers look like:

Row 0: [1, 2, 3, 0, 0] <- no delay (starts immediately)  
 Row 1: [0, 4, 5, 6, 0] <- 1 zero in front (1 cycle delay)  
 Row 2: [0, 0, 7, 8, 9] <- 2 zeros in front (2 cycle delay)

Each clock cycle, we SHIFT LEFT - the leftmost value pops out:

Shift 1 -> outputs: [1, 0, 0] Row 0 starts  
 Shift 2 -> outputs: [2, 4, 0] Row 0+1 active  
 Shift 3 -> outputs: [3, 5, 7] ALL rows active  
 Shift 4 -> outputs: [0, 6, 8] Row 0 done  
 Shift 5 -> outputs: [0, 0, 9] Only Row 2 left

### Source Code

```
// =====
// skew_buffer.v -- Input Skew Buffer for Systolic Array
// =====
//
// PROBLEM: In a systolic array, data must arrive at each PE at
// the right time. Row i of matrix A needs to be delayed by i
// clock cycles, and column j of matrix B by j cycles.
//
// SOLUTION: This module takes a FLAT matrix (all values at once)
// and outputs one skewed value per row/column per clock cycle.
//
// HOW IT WORKS:
// For N=3 and matrix A:
//   Row 0: delay=0 -> outputs a[0][0], a[0][1], a[0][2], 0, 0
//   Row 1: delay=1 -> outputs 0, a[1][0], a[1][1], a[1][2], 0
//   Row 2: delay=2 -> outputs 0, 0, a[2][0], a[2][1], a[2][2]
//
// Internally, each row has a shift register. Row i's register
// is pre-loaded with i zeros followed by the actual data.
```

```
//
// The total feeding time is 2N-1 clock cycles.
// =====

module skew_buffer #(
    parameter N = 4,                // Matrix dimension
    parameter DATA_WIDTH = 8       // Bit width of each value
)(
    input  wire clk,
    input  wire rst,
    input  wire load,                // Pulse HIGH to load matrix data
    input  wire shift_en,           // Pulse HIGH each cycle to shift data out

    // --- Matrix input (flat): NxN values loaded all at once ---
    // matrix_in[i][j] = element at row i, column j
    input  wire signed [DATA_WIDTH-1:0] matrix_in [0:N-1][0:N-1],

    // --- Skewed output: one value per row/column per clock ---
    output reg  signed [DATA_WIDTH-1:0] data_out [0:N-1]
);

// =====
// SHIFT REGISTERS
// =====
// Each row gets a shift register of length (2N-1).
// Row i is loaded as: [0, 0, ...(i zeros), a[i][0], a[i][1], ..., a[i][N-1], 0, ...]
// On each clock, we shift left and output the leftmost value.

// Shift register depth = 2*N - 1 (total cycles to feed all data)
localparam DEPTH = 2*N - 1;

reg signed [DATA_WIDTH-1:0] shift_reg [0:N-1][0:DEPTH-1];

integer r, c;

always @(posedge clk) begin
    if (rst) begin
        // Clear everything
        for (r = 0; r < N; r = r + 1) begin
            data_out[r] <= 0;
            for (c = 0; c < DEPTH; c = c + 1) begin
                shift_reg[r][c] <= 0;
            end
        end
    end
    else if (load) begin
        // Load matrix data into shift registers with skewing
        for (r = 0; r < N; r = r + 1) begin
            for (c = 0; c < DEPTH; c = c + 1) begin
                // Position (c - r) maps to matrix column
                // If c < r, it's a leading zero (delay)
                // If c >= r and c < r+N, it's actual data
                // If c >= r+N, it's a trailing zero
                if (c >= r && c < r + N) begin
                    shift_reg[r][c] <= matrix_in[r][c - r];
                end else begin
                    shift_reg[r][c] <= 0;
                end
            end
        end
    end
    // Output the first value immediately
end
```

```

        for (r = 0; r < N; r = r + 1) begin
            data_out[r] <= 0; // Will be valid after first shift
        end
    end
    else if (shift_en) begin
        // Shift left: output shift_reg[r][0], then shift everything left
        for (r = 0; r < N; r = r + 1) begin
            data_out[r] <= shift_reg[r][0];
            for (c = 0; c < DEPTH - 1; c = c + 1) begin
                shift_reg[r][c] <= shift_reg[r][c + 1];
            end
            shift_reg[r][DEPTH-1] <= 0; // Fill with zero at the end
        end
    end
end
end

// =====
// EXAMPLE: N=3, Matrix A = |1 2 3|
//                          |4 5 6|
//                          |7 8 9|
//
// After LOAD, shift registers contain:
// Row 0: [1, 2, 3, 0, 0] (no delay)
// Row 1: [0, 4, 5, 6, 0] (1 cycle delay)
// Row 2: [0, 0, 7, 8, 9] (2 cycle delay)
//
// SHIFT outputs (data_out[0], data_out[1], data_out[2]):
// Cycle 0: 1, 0, 0
// Cycle 1: 2, 4, 0
// Cycle 2: 3, 5, 7
// Cycle 3: 0, 6, 8
// Cycle 4: 0, 0, 9
//
// This is exactly the skewing pattern we need!
// =====

endmodule

```

## Key Code Explained

**localparam DEPTH = 2\*N - 1**

Total shift register length. For N=3, DEPTH=5.

### The Load Logic (line 73)

```

if (c >= r && c < r + N)
    shift_reg[r][c] <= matrix_in[r][c - r];
else
    shift_reg[r][c] <= 0;

```

This is the skewing logic:

Row 0: data at positions 0,1,2 (no delay)

Row 1: data at positions 1,2,3 (skip pos 0 = 1 cycle delay)

Row 2: data at positions 2,3,4 (skip pos 0,1 = 2 cycle delay)

### The Shift Logic (line 85-93)

When shift\_en is HIGH:

1. Output shift\_reg[r][0] (pop the leftmost value)
2. Shift everything left (move each element one position)
3. Fill the end with zero (nothing left to shift in)

## File 6: 6\_controller.v - FSM Controller

A Finite State Machine that controls WHEN things happen.

### The 4 States

```

IDLE  -----(start)-----> LOAD  -----(auto)-----> COMPUTE  -----(count done)-----> DONE
      ^                                                              |
      +------(start)-----+

```

IDLE: Waiting for start. Array held in reset (accumulators = 0)

LOAD: Load matrices into skew buffers (1 cycle)

COMPUTE: PEs are computing, data shifting through (3N-1 cycles)

DONE: Results ready! Done flag is HIGH.

### What Each State Outputs

| State   | load_en | shift_en | array_en | array_rst | done |
|---------|---------|----------|----------|-----------|------|
| IDLE    | 0       | 0        | 0        | 1         | 0    |
| LOAD    | 1       | 0        | 0        | 1         | 0    |
| COMPUTE | 0       | 1        | 1        | 0         | 0    |
| DONE    | 0       | 0        | 0        | 0         | 1    |

### Source Code

```

// =====
// controller.v -- FSM Controller for the Systolic Array
// =====
// Manages the phases of operation:
//
// IDLE --> LOAD --> COMPUTE --> DONE
//
// Timing breakdown for COMPUTE phase:
// - 1 cycle: skew buffer latency (first shift outputs appear)
// - 2N-1 cycles: data feeding from skew buffers
// - N-1 cycles: pipeline drain (last data propagates through PEs)
// Total = 1 + (2N-1) + (N-1) = 3N - 1 compute cycles
// =====

module controller #(
    parameter N = 4                // Array dimension
)(
    input  wire clk,
    input  wire rst,
    input  wire start,              // Pulse HIGH to begin computation

    output reg load_en,             // Load data into skew buffers
    output reg shift_en,            // Shift data out of skew buffers
    output reg array_en,            // Enable PEs in the systolic array
    output reg array_rst,           // Reset PEs (clear accumulators)
    output reg done                 // HIGH when results are valid
);

```

```

// State definitions
localparam STATE_IDLE    = 2'd0;
localparam STATE_LOAD    = 2'd1;
localparam STATE_COMPUTE = 2'd2;
localparam STATE_DONE    = 2'd3;

// Total compute cycles:
// - Skew buffer outputs 2N-1 values, but first output has 1-cycle latency
// - After last skew buffer output, data still needs to propagate through PEs
// - PE[N-1][N-1] receives its last data (N-1) cycles after PE[0][0]
//   due to PE pass-through delay
// Total = (2N-1) + N = 3N - 1
localparam COMPUTE_CYCLES = 3*N - 1;

reg [1:0] state, next_state;
reg [7:0] cycle_count;

// State register
always @(posedge clk) begin
    if (rst)
        state <= STATE_IDLE;
    else
        state <= next_state;
end

// Next state logic
always @(*) begin
    next_state = state;
    case (state)
        STATE_IDLE:
            if (start) next_state = STATE_LOAD;
        STATE_LOAD:
            next_state = STATE_COMPUTE;
        STATE_COMPUTE:
            if (cycle_count == COMPUTE_CYCLES - 1)
                next_state = STATE_DONE;
        STATE_DONE:
            if (start) next_state = STATE_LOAD;
        default: next_state = STATE_IDLE;
    endcase
end

// Cycle counter
always @(posedge clk) begin
    if (rst || state != STATE_COMPUTE)
        cycle_count <= 0;
    else
        cycle_count <= cycle_count + 1;
end

// Output logic
always @(*) begin
    load_en    = 1'b0;
    shift_en   = 1'b0;
    array_en   = 1'b0;
    array_rst  = 1'b0;
    done       = 1'b0;

    case (state)
        STATE_IDLE: begin

```

```

        array_rst = 1'b1;
    end
    STATE_LOAD: begin
        load_en    = 1'b1;
        array_rst = 1'b1;
    end
    STATE_COMPUTE: begin
        shift_en = 1'b1;
        array_en = 1'b1;
    end
    STATE_DONE: begin
        done = 1'b1;
    end
endcase
end
endmodule

```

## Why 3N-1 Compute Cycles?

For N=4:  $3 \times 4 - 1 = 11$  cycles. Here's why:

- 1 cycle: Skew buffer latency (first output appears)
- $2N-1 = 7$ : Feeding all skewed data
- $N-1 = 3$ : Waiting for data to propagate through last PEs
- Total:  $1 + 7 + 3 = 11$  cycles

## Key Verilog Patterns

### Three always Blocks

This FSM uses the standard 3-block pattern:

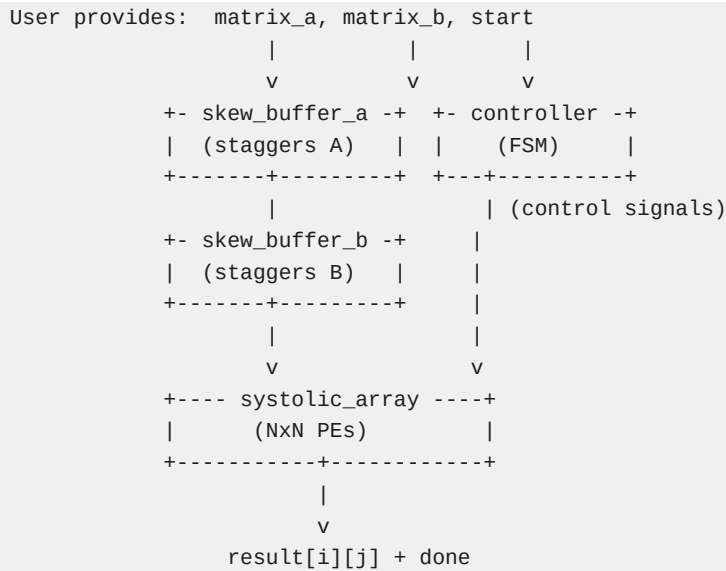
1. State Register (sequential): always @(posedge clk)  
Stores the current state. Updates on clock edge.
2. Next State Logic (combinational): always @(\*)  
Determines what state to go to next based on inputs.
3. Output Logic (combinational): always @(\*)  
Determines what outputs to drive based on current state.



## File 7: 7\_systolic\_top.v - Top-Level Wrapper

This file just **CONNECTS** the pieces together. Like LEGO.

### Block Diagram



### Source Code

```

// =====
// systolic_top.v -- Top-Level Wrapper for Systolic Array System
// =====
// Connects all the pieces together:
//
// +-----+
// |               systolic_top               |
// |                                           |
// | +-----+                               |
// | | Controller |--> load_en, shift_en,    |
// | |   FSM     |   array_en, array_rst, done |
// | +-----+                               |
// | |           |                               |
// | +-----v-----+ +-----+             |
// | | Skew Buf A |-->|           |           |
// | | (rows)      | |   NxN      |           |
// | +-----+    | |   Systolic |--> result |
// | +-----+    | |   Array     |           |
// | | Skew Buf B |-->|           |           |
// | | (columns)   | +-----+             |
// | +-----+                               |
// +-----+
//
// USAGE:
// 1. Load matrix_a and matrix_b on the inputs
// 2. Pulse 'start' HIGH for one cycle
// 3. Wait for 'done' to go HIGH
// 4. Read results from result[i][j]
// =====
  
```

```

module systolic_top #(
    parameter N = 4,                // Matrix dimension (NxN)
    parameter DATA_WIDTH = 8       // Bit width of each element
)(
    input  wire clk,
    input  wire rst,
    input  wire start,              // Pulse to begin computation

    // --- Input matrices (flat NxN arrays) ---
    input  wire signed [DATA_WIDTH-1:0] matrix_a [0:N-1][0:N-1],
    input  wire signed [DATA_WIDTH-1:0] matrix_b [0:N-1][0:N-1],

    // --- Output: result matrix C = A x B ---
    output wire signed [2*DATA_WIDTH-1:0] result [0:N-1][0:N-1],

    // --- Status ---
    output wire done
);

// =====
// INTERNAL SIGNALS
// =====

// Controller outputs
wire load_en;
wire shift_en;
wire array_en;
wire array_rst;

// Skewed data outputs (fed into the systolic array)
wire signed [DATA_WIDTH-1:0] a_skewed [0:N-1]; // One per row
wire signed [DATA_WIDTH-1:0] b_skewed [0:N-1]; // One per column

// Transposed B matrix for column-wise skewing
// The B skew buffer skews by ROW, but we need B skewed by COLUMN.
// So we transpose B first: b_transposed[j][i] = matrix_b[i][j]
// Then skewing by row on the transposed matrix = skewing by column on the original.
reg signed [DATA_WIDTH-1:0] matrix_b_transposed [0:N-1][0:N-1];

integer ti, tj;
always @(*) begin
    for (ti = 0; ti < N; ti = ti + 1)
        for (tj = 0; tj < N; tj = tj + 1)
            matrix_b_transposed[ti][tj] = matrix_b[tj][ti];
end

// =====
// CONTROLLER FSM
// =====
controller #(
    .N(N)
) ctrl (
    .clk      (clk),
    .rst      (rst),
    .start    (start),
    .load_en  (load_en),
    .shift_en (shift_en),
    .array_en (array_en),
    .array_rst(array_rst),
    .done     (done)

```

```

);

// =====
// SKEW BUFFER FOR MATRIX A (rows)
// =====
// A values flow LEFT -> RIGHT through the array
// Row i is delayed by i clock cycles
skew_buffer #(
    .N(N),
    .DATA_WIDTH(DATA_WIDTH)
) skew_a (
    .clk      (clk),
    .rst      (rst),
    .load     (load_en),
    .shift_en (shift_en),
    .matrix_in (matrix_a),
    .data_out  (a_skewed)
);

// =====
// SKEW BUFFER FOR MATRIX B (columns)
// =====
// B values flow TOP -> BOTTOM through the array
// Column j is delayed by j clock cycles
// We use the TRANSPOSED B so that row-skewing = column-skewing
skew_buffer #(
    .N(N),
    .DATA_WIDTH(DATA_WIDTH)
) skew_b (
    .clk      (clk),
    .rst      (rst),
    .load     (load_en),
    .shift_en (shift_en),
    .matrix_in (matrix_b_transposed),
    .data_out  (b_skewed)
);

// =====
// SYSTOLIC ARRAY (NxN PEs)
// =====
systolic_array #(
    .N(N),
    .DATA_WIDTH(DATA_WIDTH)
) array_inst (
    .clk      (clk),
    .rst      (array_rst),
    .en       (array_en),
    .a_in     (a_skewed),
    .b_in     (b_skewed),
    .result   (result)
);

endmodule

```

## The B Matrix Transpose Trick

The skew buffer staggers data by ROWS.  
But B needs to be staggered by COLUMNS.

Solution: Transpose B (swap rows and columns), THEN feed it to the same skew buffer.

Transposing B: `b_transposed[j][i] = matrix_b[i][j]`

Row-staggering the transposed B = Column-staggering the original B

This lets us REUSE the same skew\_buffer module for both A and B!

### Usage

1. Load matrix\_a and matrix\_b on the inputs
2. Pulse 'start' HIGH for one cycle
3. Wait for 'done' to go HIGH
4. Read results from `result[i][j]`

## File 8: 8\_systolic\_4x4\_tb.v - Full System Test

Tests the COMPLETE system with 3 different matrix multiplications.

### Structure Overview

This is 294 lines, but it's really just 3 tests back to back:

Lines 1-60: Setup (signals, instantiate systolic\_top)  
 Lines 62-120: Helper tasks (compute\_expected, check\_results)  
 Lines 140-187: TEST 1:  $A \times \text{Identity} = A$   
 Lines 189-217: TEST 2:  $A \times 2I = 2A$   
 Lines 219-275: TEST 3: General 4x4 multiply

### The 3 Tests

#### Test 1: $A \times I = A$ (Identity)

Multiply A by the identity matrix. Result should equal A.  
 This is the simplest sanity check - if this fails, something is fundamentally wrong.

#### Test 2: $A \times 2I = 2A$ (Scalar)

Multiply A by 2\*Identity. Every element should double.  
 Tests that the accumulation works correctly.

#### Test 3: General Multiply

Two arbitrary matrices. Software computes the expected result, hardware computes the actual result, and we compare.

### Source Code

```
// =====
// systolic_4x4_tb.v -- Full System Testbench (4x4 matrices)
// =====
// Tests the COMPLETE system: top-level wrapper with controller,
// skew buffers, and 4x4 systolic array.
//
// Test matrices:
//   A = | 1  2  3  4|   B = | 1  0  0  0|
//       | 5  6  7  8|       | 0  1  0  0|
//       | 9 10 11 12|       | 0  0  1  0|
//       |13 14 15 16|       | 0  0  0  1|
//
// B is the IDENTITY matrix, so C = A x B = A
// This makes verification trivial!
//
// Then we also test with non-trivial matrices:
//   A = |1 2 3 4|   B = |2 0 0 0|
//       |5 6 7 8|       |0 2 0 0|
```

```

//      |0 0 0 0|      |0 0 2 0|
//      |1 1 1 1|      |0 0 0 2|
//
// C = A x 2I = 2A (every element doubled)
// =====

`timescale 1ns / 1ps

module systolic_4x4_tb;

    // ---- Parameters ----
    parameter N = 4;
    parameter DATA_WIDTH = 8;
    parameter CLK_PERIOD = 10;

    // ---- Signals ----
    reg clk, rst, start;
    reg signed [DATA_WIDTH-1:0] matrix_a [0:N-1][0:N-1];
    reg signed [DATA_WIDTH-1:0] matrix_b [0:N-1][0:N-1];
    wire signed [2*DATA_WIDTH-1:0] result [0:N-1][0:N-1];
    wire done;

    // Expected result storage
    reg signed [2*DATA_WIDTH-1:0] expected [0:N-1][0:N-1];

    // ---- Instantiate the Top-Level ----
    systolic_top #(
        .N(N),
        .DATA_WIDTH(DATA_WIDTH)
    ) uut (
        .clk(clk),
        .rst(rst),
        .start(start),
        .matrix_a(matrix_a),
        .matrix_b(matrix_b),
        .result(result),
        .done(done)
    );

    // ---- Clock Generator ----
    initial clk = 0;
    always #(CLK_PERIOD/2) clk = ~clk;

    // ---- Dump waveforms ----
    initial begin
        $dumpfile("systolic_4x4_tb.vcd");
        $dumpvars(0, systolic_4x4_tb);
    end

    // ---- Helper: compute expected result in software ----
    task compute_expected;
        integer i, j, k;
        begin
            for (i = 0; i < N; i = i + 1)
                for (j = 0; j < N; j = j + 1) begin
                    expected[i][j] = 0;
                    for (k = 0; k < N; k = k + 1)
                        expected[i][j] = expected[i][j] + matrix_a[i][k] * matrix_b[k][j];
                end
            end
        end
    endtask

```

```
// ---- Helper: check results ----
integer pass_count;
integer fail_count;
task check_results;
    input [8*20-1:0] test_name; // String name for the test
    integer i, j;
    begin
        pass_count = 0;
        fail_count = 0;
        $display("\n Results for %0s:", test_name);
        $display(" +-----+");
        for (i = 0; i < N; i = i + 1) begin
            $write(" | ");
            for (j = 0; j < N; j = j + 1) begin
                if (result[i][j] == expected[i][j]) begin
                    $write("%5d ", result[i][j]);
                    pass_count = pass_count + 1;
                end else begin
                    $write("%5d!", result[i][j]);
                    fail_count = fail_count + 1;
                end
            end
            $display("|");
        end
        $display(" +-----+");

        if (fail_count == 0)
            $display(" >>> PASS! All %0d elements correct <<<", pass_count);
        else
            $display(" >>> FAIL! %0d/%0d elements wrong <<<", fail_count, pass_count + fail_count);
    end
endtask

// ---- Helper: print matrix ----
task print_matrix;
    input [8*20-1:0] name;
    integer i, j;
    begin
        $display(" %0s:", name);
        for (i = 0; i < N; i = i + 1) begin
            $write(" | ");
            for (j = 0; j < N; j = j + 1)
                $write("%3d ", matrix_a[i][j]);
            $display("|");
        end
    end
endtask

// ---- Main Test Sequence ----
integer i, j;
initial begin
    $display("=====");
    $display(" 4x4 Systolic Array -- Full System Test");
    $display("=====");

    // ---- RESET ----
    rst = 1;
    start = 0;
    for (i = 0; i < N; i = i + 1)
        for (j = 0; j < N; j = j + 1) begin
```

```

        matrix_a[i][j] = 0;
        matrix_b[i][j] = 0;
    end

repeat(3) @(posedge clk);
rst = 0;
@(posedge clk);

// =====
// TEST 1: A x Identity = A
// =====
$display("\n-----");
$display("  TEST 1: A x I = A (identity matrix test)");
$display("-----");

// Load A = |1 2 3 4| |5 6 7 8| |9 10 11 12| |13 14 15 16|
matrix_a[0][0]=1; matrix_a[0][1]=2; matrix_a[0][2]=3; matrix_a[0][3]=4;
matrix_a[1][0]=5; matrix_a[1][1]=6; matrix_a[1][2]=7; matrix_a[1][3]=8;
matrix_a[2][0]=9; matrix_a[2][1]=10; matrix_a[2][2]=11; matrix_a[2][3]=12;
matrix_a[3][0]=13; matrix_a[3][1]=14; matrix_a[3][2]=15; matrix_a[3][3]=16;

// Load B = Identity
matrix_b[0][0]=1; matrix_b[0][1]=0; matrix_b[0][2]=0; matrix_b[0][3]=0;
matrix_b[1][0]=0; matrix_b[1][1]=1; matrix_b[1][2]=0; matrix_b[1][3]=0;
matrix_b[2][0]=0; matrix_b[2][1]=0; matrix_b[2][2]=1; matrix_b[2][3]=0;
matrix_b[3][0]=0; matrix_b[3][1]=0; matrix_b[3][2]=0; matrix_b[3][3]=1;

compute_expected;

// Start computation
start = 1;
@(posedge clk);
start = 0;

// Wait for done
wait(done == 1);
@(posedge clk);

check_results("A x Identity");

$display("\n  Expected (should equal A):");
for (i = 0; i < N; i = i + 1) begin
    $write("    | ");
    for (j = 0; j < N; j = j + 1)
        $write("%5d ", expected[i][j]);
    $display("|");
end

// =====
// TEST 2: A x 2I = 2A (scalar multiply test)
// =====
$display("\n-----");
$display("  TEST 2: A x 2I = 2A (double every element)");
$display("-----");

// Keep A the same, change B to 2*Identity
matrix_b[0][0]=2; matrix_b[0][1]=0; matrix_b[0][2]=0; matrix_b[0][3]=0;
matrix_b[1][0]=0; matrix_b[1][1]=2; matrix_b[1][2]=0; matrix_b[1][3]=0;
matrix_b[2][0]=0; matrix_b[2][1]=0; matrix_b[2][2]=2; matrix_b[2][3]=0;
matrix_b[3][0]=0; matrix_b[3][1]=0; matrix_b[3][2]=0; matrix_b[3][3]=2;

```



```

compute_expected;

// Reset and restart -- need enough cycles for array_rst to clear accumulators
rst = 1;
repeat(3) @(posedge clk);
rst = 0;
@(posedge clk);

start = 1;
@(posedge clk);
start = 0;

wait(done == 1);
@(posedge clk);

check_results("A x 2I");

// =====
// TEST 3: General matrix multiplication
// =====
$display("\n-----");
$display("  TEST 3: General 4x4 matrix multiply");
$display("-----");

matrix_a[0][0]=1; matrix_a[0][1]=2; matrix_a[0][2]=0; matrix_a[0][3]=1;
matrix_a[1][0]=0; matrix_a[1][1]=1; matrix_a[1][2]=3; matrix_a[1][3]=0;
matrix_a[2][0]=2; matrix_a[2][1]=0; matrix_a[2][2]=1; matrix_a[2][3]=4;
matrix_a[3][0]=1; matrix_a[3][1]=1; matrix_a[3][2]=1; matrix_a[3][3]=1;

matrix_b[0][0]=2; matrix_b[0][1]=1; matrix_b[0][2]=0; matrix_b[0][3]=3;
matrix_b[1][0]=0; matrix_b[1][1]=3; matrix_b[1][2]=1; matrix_b[1][3]=0;
matrix_b[2][0]=1; matrix_b[2][1]=0; matrix_b[2][2]=2; matrix_b[2][3]=1;
matrix_b[3][0]=3; matrix_b[3][1]=2; matrix_b[3][2]=0; matrix_b[3][3]=1;

compute_expected;

$display("  Matrix A:");
for (i = 0; i < N; i = i + 1) begin
    $write("    | ");
    for (j = 0; j < N; j = j + 1)
        $write("%3d ", matrix_a[i][j]);
    $display("|");
end
$display("  Matrix B:");
for (i = 0; i < N; i = i + 1) begin
    $write("    | ");
    for (j = 0; j < N; j = j + 1)
        $write("%3d ", matrix_b[i][j]);
    $display("|");
end
$display("  Expected C = A x B:");
for (i = 0; i < N; i = i + 1) begin
    $write("    | ");
    for (j = 0; j < N; j = j + 1)
        $write("%5d ", expected[i][j]);
    $display("|");
end

// Reset and restart -- need enough cycles for array_rst to clear accumulators
rst = 1;
repeat(3) @(posedge clk);

```

```

rst = 0;
@(posedge clk);

start = 1;
@(posedge clk);
start = 0;

wait(done == 1);
@(posedge clk);

check_results("General 4x4");

// =====
// SUMMARY
// =====
$display("\n=====");
$display("  ALL TESTS COMPLETE!");
$display("=====\\n");

repeat(3) @(posedge clk);
$finish;
end

// ---- Timeout watchdog ----
initial begin
  #100000;
  $display("ERROR: Simulation timed out!");
  $finish;
end

endmodule

```

## Key Construct: Verilog Tasks

A 'task' in Verilog is like a function - reusable code.

**compute\_expected:** Does matrix multiplication in software  
(using for loops) to get the 'right answer'

**check\_results:** Compares hardware output vs software answer  
and prints PASS or FAIL for each element

## Each Test Pattern

```

// 1. Set matrix values
matrix_a[0][0] = 1; ...
matrix_b[0][0] = 5; ...

// 2. Compute expected answer (software golden model)
compute_expected;

// 3. Reset the hardware
rst = 1;
repeat(3) @(posedge clk);
rst = 0;

// 4. Start computation

```

```
start = 1;
@(posedge clk);
start = 0;

// 5. Wait for hardware to finish
wait(done == 1);

// 6. Check results
check_results("Test Name");
```

```
RUN: iverilog -g2012 -o test.vvp 1_pe.v 3_systolic_array.v 5_skew_buffer.v 6_controller.v 7_systolic_top.v 8_systolic_
```

## Quick Reference: Simulation Commands

### Test 1: Single PE

```
iverilog -o pe_test.vvp 1_pe.v 2_pe_tb.v
vvp pe_test.vvp
gtkwave pe_tb.vcd
```

### Test 2: 2x2 Array (Manual Skewing)

```
iverilog -g2012 -o array_2x2_test.vvp 1_pe.v 3_systolic_array.v 4_systolic_2x2_tb.v
vvp array_2x2_test.vvp
gtkwave systolic_2x2_tb.vcd
```

### Test 3: Full 4x4 System

```
iverilog -g2012 -o full_test.vvp 1_pe.v 3_systolic_array.v 5_skew_buffer.v 6_controller.v 7_systolic_top.v 8_systolic_4x4_tb.v
vvp full_test.vvp
gtkwave systolic_4x4_tb.vcd
```

### GTKWave Tips

1. Expand the module tree in the left panel
2. Select signals, press Insert (or click Append)
3. Right-click signal -> Data Format -> Signed Decimal
4. Use Ctrl+Shift+F to zoom to fit all data
5. Watch 'acc' signal grow as the PE accumulates

### Terminology Cheat Sheet

| Term              | Meaning                                              |
|-------------------|------------------------------------------------------|
| PE                | Processing Element - one compute unit, does MAC      |
| MAC               | Multiply-Accumulate: $acc = acc + a \times b$        |
| Systolic          | Data flows rhythmically, like a heartbeat            |
| Skewing           | Staggering input data for correct timing             |
| Output Stationary | Results stay in PEs, inputs flow through             |
| generate          | Verilog construct to create hardware at compile time |
| always @(posedge) | Execute on every rising clock edge                   |
| <=                | Non-blocking assignment (for sequential logic)       |
| \$dumpfile        | Create waveform file for GTKWave                     |
| localparam        | Compile-time constant (cannot be overridden)         |